

Datenmanagement

Dateipfade

1. Dokumentieren Sie alle in dieser Einheit verwendeten R-Befehle in einem R-Skript.
2. Verwenden Sie den Befehl `sys.frame(1)$ofile`, um den absoluten Pfad des aktuellen Skripts zu ermitteln, und speichern Sie diesen in einer Variablen `skript_pfad`. Definieren Sie mithilfe dieser Variablen eine neue Variable `skript_ordner`, die nur den Pfad zum Verzeichnis des Skripts (ohne Dateinamen) enthält. Dieser Ordner wird im Folgenden als *Skriptordner* bezeichnet. Geben Sie den absoluten Pfad des Skriptordners aus.
3. Ermitteln Sie das aktuelle Arbeitsverzeichnis (working directory) mit der Funktion `getwd()` und speichern Sie den Pfad in einer Variablen `current_wd`. Ändern Sie anschließend das Arbeitsverzeichnis mit der Funktion `setwd()` auf den Skriptordner (verwenden Sie dazu die Variable `skript_ordner`). Geben Sie das neue Arbeitsverzeichnis zur Überprüfung aus.
4. Ändern Sie Ihr Arbeitsverzeichnis zu einem anderen Ordner Ihrer Wahl (z.B. zum übergeordneten Verzeichnis mit `setwd("../")`). Überprüfen Sie mit `getwd()`, ob die Änderung erfolgreich war. Setzen Sie das Arbeitsverzeichnis anschließend wieder auf den ursprünglichen Skriptordner zurück und überprüfen Sie erneut, ob die Rücksetzung funktioniert hat.

Daten einlesen und daten speichern

1. Stellen Sie sicher, dass Ihr Arbeitsverzeichnis auf den Skriptordner gesetzt ist. Lesen Sie die Datei `Beispieldatensatz.csv` mit der Funktion `read.csv()` in ein Dataframe ein. Speichern Sie den Dateipfad zunächst in einer Pfadvariablen (z.B. `daten_pfad`) und verwenden Sie diese Variable beim Einlesen. Geben Sie die ersten Zeilen des eingelesenen Dataframes mit `head()` aus.
2. Fügen Sie dem eingelesenen Dataframe eine neue Spalte oder eine neue Zeile mit selbst gewählten Werten hinzu. Speichern Sie das erweiterte Dataframe als neue CSV-Datei mit dem Namen `Beispieldatensatz_erweitert.csv` im Skriptordner. Verwenden Sie auch hier eine Pfadvariable (z.B. `daten_erweitert_pfad`). Überprüfen Sie, ob die Datei erfolgreich gespeichert wurde, indem Sie sie erneut einlesen und ausgeben.
3. Ändern Sie Ihr Arbeitsverzeichnis zum übergeordneten Verzeichnis (parent directory) mit `setwd("../")`. Passen Sie nun Ihre Pfadvariablen aus Aufgabe 1 und 2 so an, dass sie relative Pfade verwenden, die vom neuen Arbeitsverzeichnis aus auf die Dateien im Skriptordner verweisen. Führen Sie das Einlesen und Speichern erneut aus, um zu überprüfen, ob die angepassten Pfade korrekt funktionieren.
 - *Hinweis:* Wenn Ihr Skriptordner z.B. `kurs/uebungen` heißt und Sie nun im (parent) Ordner `kurs` sind, müssten die Pfadvariablen etwa so aussehen: `daten_pfad <- uebungen/Beispieldatensatz.csv`
 - Beachten Sie, dass sich in Ihrer Ordnerstruktur auf der Festplatte nichts ändert, sondern lediglich das Arbeitsverzeichnis und die Pfadvariablen angepasst werden.

Datensimulation

- Der folgende Code generiert zwei Vektoren, die zwei Variablen eines Datensatzes mit Stichprobengröße $n = 30$ repräsentieren. Die erste Variable `var_1` enthält Gruppenzugehörigkeiten („Winter“ und „Sommer“) mit den gruppenspezifischen Stichprobengrößen `n_1` beziehungsweise `n_2`. Die zweite Variable `var_2` enthält Zahlenwerte.

```
n      <- 30
n_1    <- (n / 3) * 2
n_2    <- (n / 3) * 1

var_1 <- c(
  rep("Winter", n_1),
  rep("Sommer", n_2)
)

set.seed(0)
var_2 <- c(
  runif(n_1, min = 0, max = 5),
  runif(n_2, min = 5, max = 9)
)
```

Passen Sie den Code so an, dass beide Vektoren jeweils 150 Datenpunkte enthalten. Die erste Hälfte der Datenpunkte repräsentiert die Gruppe „Winter“, die zweite Hälfte die Gruppe „Sommer“. Gehen Sie dabei wie folgt vor:

- Setzen Sie zu Beginn des Codes den Zufallsgenerator-Seed mit `set.seed(0)`. Dadurch wird sichergestellt, dass der Zufallsgenerator bei jedem Ausführen des Codes die gleichen Zahlen generiert, sodass die Ergebnisse reproduzierbar sind.
- Verwenden Sie die Funktion `runif()`, um Zufallszahlen für die Variable `var_2` zu generieren (*Hinweis:* Nutzen Sie `?runif`, um die Funktion und ihre Argumente zu verstehen).
- Passen Sie die Argumente im oben gegebenen Code so an, dass für die Gruppe „Winter“ Werte zwischen 0 und 10 und für die Gruppe „Sommer“ Werte zwischen 0 und 40 generiert werden.
- Runden Sie alle Zahlenwerte auf eine Nachkommastelle mit der Funktion `round()`.
- Geben Sie beide Vektoren mit `print()` aus.

Optional: Überlegen Sie sich ein passendes Beispiel, was die Variable `var_2` darstellen könnte.

- Erstellen Sie ein Dataframe aus den beiden Vektoren `var_1` und `var_2`. Vergeben Sie sinnvolle Spaltennamen (z.B. `Jahreszeit` und `Messwert`). Geben Sie die ersten Zeilen des Dataframes mit `head()` aus und speichern Sie es als CSV-Datei im Skriptordner. Benennen Sie die Datei: `simulierte_saisondaten.csv`